Reinforcement Learning for the Continuous Art Gallery Problem

Preliminaries, Policy Architecture, and MDP Formulation

AGP Project — Technical Report

November 2, 2026

Abstract

We train a PPO agent to place point guards in polygonal galleries with holes so that at least 98% of the free space is seen, using as few guards as possible. This report explains every design decision in the current implementation. Section 2 gives the geometry that motivates a two-stage curriculum: coverage is monotone submodular, so a guard's redundancy can only be measured *after* the gallery is saturated, and the dense rewards we use are potential-based, so they telescope into the true objective. Section 3 describes the policy network and why each part of it exists: it keeps the map's spatial layout to the output and factors each placement into a choice of region and a *continuous* point inside it, and it masks the three moves that are never right (STOP or DEL below the coverage threshold, STOP while a guard can still go). Section 4 states the MDP and the reward with the exact constants used in code. Section 5 lists the settings that were chosen rather than tuned and how to test them, and how to decide whether a relocation action is needed.

Contents

1 Introduction **2**

2 Preliminaries: why two stages **2**
2.1 Galleries, visibility, coverage . . . 2
2.2 Complexity, and what a learned policy can gain . . . 3
2.3 Submodularity: why pruning must come second . . . 3
2.4 Greedy, its guarantees, and the reference count . . . 4
2.5 Why one stage collapses . . . 5
2.6 Potential-based shaping: dense reward without bias . . . 5

3 Architecture **6**
3.1 What the policy has to do . . . 6
3.2 The network . . . 7
3.3 The factored action . . . 8
3.4 The mode head across the stage switch (R5) . . . 11
3.5 PPO configuration . . . 11
3.6 Results . . . 13

4 MDP formulation and reward **15**
4.1 Instances . . . 16
4.2 State, actions, termination . . . 16
4.3 Observation . . . 17
4.4 Reward . . . 17

4.5 Schedule . . . 19
4.6 Weight calibration . . . 19

5 Open settings and a possible relocation move **20**
5.1 Settings that were chosen, not tuned . . . 20
5.2 Is a relocation move needed? . . . 20

1 Introduction

The agent sees an image of a gallery and the guards placed so far. At each step it either adds a guard at a point, deletes the guard nearest to a point, or stops. Training runs in two stages on one network (fig. 3):

Saturate (stage 1). Only ADD is allowed and guards are free. The agent learns to cover arbitrary galleries.

Prune (stage 2). DEL and STOP are unmasked, every guard costs reward, and deleting a *free* guard, one whose loss keeps the gallery covered, is paid. The agent learns to place fewer guards and to remove the ones it does not need.

Three facts shape everything below. The problem is NP-hard, so a learned heuristic is a reasonable target (section 2.2). Coverage is submodular, which fixes the *order* of the two stages (section 2.3). And guards see without a range limit, so whether a point is a good guard position depends on the whole gallery, which shapes the network (section 3). The implementation lives in `agp/`: geometry in `gallery.py`, the MDP in `env.py`, rewards in `rewards.py`, the policy in `policy.py`, and the curriculum in `train.py`. All constants quoted below are the values in that code.

2 Preliminaries: why two stages

2.1 Galleries, visibility, coverage

Definition 1 (Gallery and free space)**.** A gallery $G = (P, \mathcal{H})$ is a simple polygon $P \subset \mathbb{R}^2$ with a finite set $\mathcal{H}$ of pairwise disjoint polygonal holes inside it. The free space is $\mathcal{F} = P \setminus \bigcup_{H \in \mathcal{H}} \operatorname{int} H$, with area $A = |\mathcal{F}| > 0$.

Definition 2 (Visibility polygon)**.** A guard at $g \in \mathcal{F}$ sees $\mathcal{V}(g) = \{q \in \mathcal{F} : \overline{gq} \subseteq \mathcal{F}\}$.

Visibility has *unlimited range*: a guard sees every point it has an unobstructed segment to, however far, and only the outer wall and the holes block it (fig. 1). `Gallery.visibility_polygon` computes $\mathcal{V}(g)$ exactly. It casts one ray towards every vertex of P and of every hole, plus one ray $\pm 10^{-6}$ rad to either side, so that both edges meeting at an occluding (reflex) corner are captured. Each ray is $3\times$ the bounding-box diagonal long, so effectively infinite, and is cut at its first boundary hit. The hits, taken in angular order, form $\mathcal{V}(g)$, which is clipped back to $\mathcal{F}$.

Definition 3 (Coverage)**.** For a finite guard set $S \subset \mathcal{F}$ let $\mathcal{C}(S) = \bigcup_{g \in S} \mathcal{V}(g)$ and $f(S) = |\mathcal{C}(S)|/A \in [0, 1]$. A gallery is *solved* when $f(S) \geq c^\dagger$ with $c^\dagger = 0.98$.

The threshold $c^\dagger < 1$ is deliberate. Full coverage can require a guard for a sliver of a few square units behind a hole, and that last guard dominates the count without telling us anything about placement quality. At 98% the greedy baseline solves 598/600 training and 88/90 test galleries under the 16-guard budget; the rest stop between 97.7% and 98% because no candidate point adds new area (section 2.4).

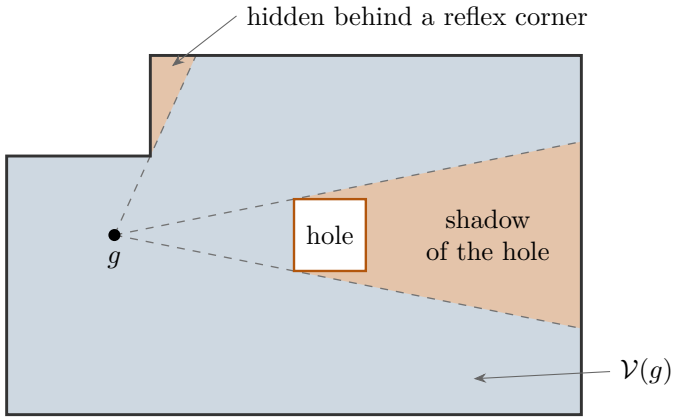


Figure 1: Visibility has unlimited range and is blocked only by walls and holes. The guard g sees the whole blue region, up to the far wall. It cannot see the orange regions: the shadow cast by the hole, bounded by the rays through the hole's two silhouette corners, and a thin wedge behind the reflex corner of the outer wall. Thin wedges like this are what the last few guards of a solution exist to cover.

Two representations of the same geometry. Every quantity that enters the *reward* is computed exactly with Shapely polygons: $f(S)$, the area a new guard adds, and the area lost when a guard is removed. The *observation* is a 64×64 raster of the gallery's bounding box: pixel (r, c) samples the point $(x_{\min} + c\,w/63,\ y_{\min} + r\,h/63)$, where w, h are the box's width and height. The raster also gives a cheap estimate of each guard's exclusive area (section 4.3), used to screen which guards need an exact check. Exact rewards mean the agent is never paid for raster artefacts; a raster observation means the network can be convolutional.

2.2 Complexity, and what a learned policy can gain

Chvátal's theorem gives $\lfloor n/3 \rfloor$ vertex guards as the tight worst case for a simple n-gon [4, 8]. Finding a *minimum* guard set is NP-hard [10], hard to approximate [6], and with guards allowed anywhere in the polygon the decision problem is $\exists\mathbb{R}$-complete [1]. Exact and near-exact solvers exist but restrict guards to finite candidate sets such as vertices or grid points [5, 15].

This tells us where a learned policy can do better than classical heuristics:

1. **Continuous positions.** Optimal point guards need not lie on any candidate set. A policy that outputs real coordinates can reach positions no candidate-based method can. We keep the action continuous for this reason (section 3.3).
2. **Placement.** Greedy never revisits a choice, so a guard placed early for its large gain can force awkward extra guards later. A policy that learns where to place can use fewer guards from the start.
3. **Pruning.** Guards that became free as later guards overlapped them can be removed. In practice this matters less than placement: greedy's own prune step removes a guard in only 59 of 600 training galleries, and the learned policy's solutions have similarly few free guards (section 3.6).

2.3 Submodularity: why pruning must come second

Proposition 4 (Monotone submodularity)**.** *$f(\emptyset) = 0$, f is monotone, and for all $S \subseteq S'$ and every g,*

$$f(S \cup \{g\}) - f(S) \ \geq\ f(S' \cup \{g\}) - f(S').$$

Proof. With $\nu = |\cdot|/A$, the marginal gain is $\nu(\mathcal{V}(g) \setminus \mathcal{C}(S))$. Since $\mathcal{C}(S) \subseteq \mathcal{C}(S')$, the set being measured can only shrink as S grows. □

Two quantities drive the design. For a candidate $g \notin S$ and a placed guard $g_j \in S$,

$$\delta^+(g \mid S) = f(S \cup \{g\}) - f(S), \qquad \delta^-(g_j \mid S) = f(S) - f(S \setminus \{g_j\}).$$

δ^+ is the *marginal gain* of adding, and δ^- is the *exclusive area* of a placed guard, i.e. what deleting it would cost. A guard with $\delta^- = 0$ is strictly redundant.

Proposition 5 (Redundancy is revealed only after saturation)**.** *Insert guards $g_1, \ldots, g_K$ in order and let $S_j = \{g_1, \ldots, g_j\}$. Then for every j,*

$$\delta^-(g_j \mid S_K) \;\le\; \delta^+(g_j \mid S_{j-1}),$$

and $\delta^-(g_j \mid S)$ is non-increasing in $S \ni g_j$.

Proof. Apply proposition 4 to $S_{j-1} \subseteq S_K \setminus \{g_j\}$. □

Figure 2 shows both propositions on a real test gallery. The coverage curve flattens as guards are added, and the second guard, which added 28% of the gallery when it was placed, ends up with an exclusive area of zero.

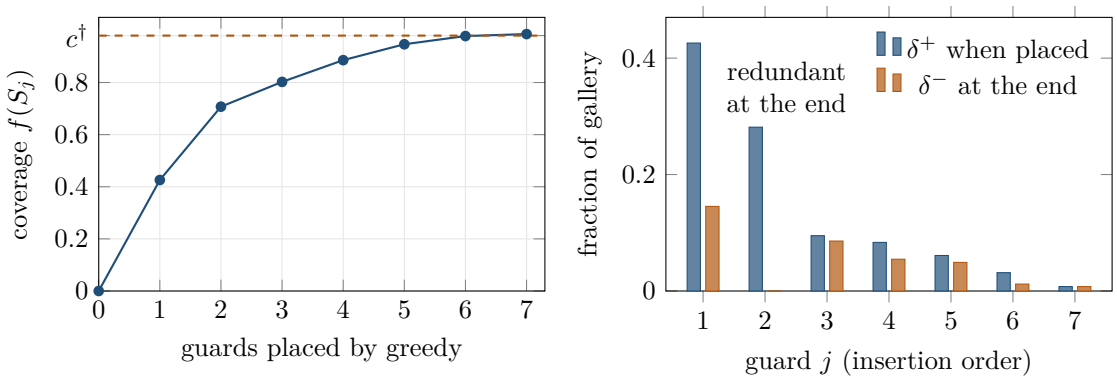


Figure 2: Greedy saturates with diminishing returns and leaves redundant guards behind. Real data: greedy cover of hard test gallery #63. *Left:* each extra guard adds less (proposition 4); the seventh adds 0.8% to cross $c^\dagger$. *Right:* for every guard, the exclusive area at the end (δ^-) is at most its gain when placed (δ^+), as proposition 5 requires. Guard 2 covered 28% when placed and is fully redundant at the end, so the greedy prune removes it ($k_{\text{greedy}} = 7$, $k_{\text{ref}} = 6$). Its redundancy only became visible after guards 3–7 were placed.

Consequence. A guard can only become *cheaper* to delete as more guards are added. A deletion decision made early in an episode sees an exclusive area that is too large; after saturation it sees the smallest value that guard will ever have. Learning to cover first and to prune second is therefore the order in which the pruning signal is most informative, not just a convenience.

2.4 Greedy, its guarantees, and the reference count

The greedy rule "add the guard of largest δ^+" inherits the $(1 - 1/e)$ guarantee for budgeted monotone submodular maximisation [13] and the logarithmic-factor guarantee for full cover, which is essentially the best possible for set cover in polynomial time [7]. Greedy is therefore strong at *covering*. It is weak at *minimality*: by proposition 5, guards chosen early lose exclusive area as later guards overlap them, and greedy never revisits a choice.

Our baseline (`baselines.py`) makes this concrete for each gallery:

1. **Candidates.** One point per vertex of P and of every hole, moved 1% of the diagonal towards an interior reference point, plus 40 uniform interior samples.
2. **Greedy cover.** Repeatedly add the candidate with the largest raster marginal gain until $f \ge c^\dagger$, the budget of 16 is reached, or no candidate adds area. The count is k_{greedy}.

3. **Greedy prune.** Repeatedly delete the guard with the smallest exact δ^- while coverage stays $\geq c^\dagger$. The remaining count is k_{ref}.

k_{ref} is the per-instance reference that the parsimony reward and the reported *guard ratio* k/k_{ref} are measured against. It is an *upper bound* on the optimum, not the optimum: it depends on the candidate set, and the true minimum is NP-hard to compute. A guard ratio below 1 means beating greedy, not proving optimality. On the training set, k_{greedy} ranges over 1–3 (easy), 2–5 (mid) and 3–11 (hard). Mean test-set values are in table 1.

Table 1: Greedy baseline on the 90 test galleries (30 per tier).

tier	mean k_{greedy}	mean k_{ref} (after prune)	mean coverage
easy	1.93	1.93	0.994
mid	2.70	2.67	0.992
hard	6.07	5.97	0.986

2.5 Why one stage collapses

The objective is to solve the gallery with few guards. With a guard cost w_{par} per guard and a coverage weight w_{cov}, the immediate value of one more ADD is $w_{\text{cov}}\,\delta^+ - w_{\text{par}}$, which is negative whenever

$$\delta^+ \;<\; w_{\text{par}}/w_{\text{cov}} \;=\; 0.15 \quad \text{(our stage-2 weights)}.$$

In fig. 2, guards 3–7 all add less than 15%. Their only justification is the terminal bonus for crossing $c^\dagger$, and a policy that cannot yet cross $c^\dagger$ reliably never collects it. From such a policy's point of view the late guards are pure cost, so it learns to place fewer of them, which makes crossing $c^\dagger$ rarer still. The fixed point of this feedback is "place few guards and fail".

Stage 1 breaks the loop by setting $w_{\text{par}} = 0$: every ADD has non-negative value, the goal bonus is reachable, and the agent learns to cover. Stage 2 turns the cost on only after covering is reliable (fig. 3). This matches the reference locomotion work, where a single-stage dribbling policy reaches 33% success against 86.7% with two stages [16].

2.6 Potential-based shaping: dense reward without bias

PPO needs a dense signal, but dense rewards can change which policy is optimal, and a reward paid per *event* can be farmed by repeating the event. We avoid both by paying every dense term as the change of a function of the state. Let $u(s) = 1 - f(S)$, $k(s) = |S|$, and let

$$R(s) \;=\; \big|\{\, g_j \in S \;:\; f(S) - \delta^-(g_j \mid S) \geq c^\dagger \,\}\big|$$

be the number of *free* guards: those that could each be deleted on their own with the gallery still solved. "Free" is the success rule itself, so a guard alone covering 1.2% of a gallery at 99.5% coverage is free, while one covering 0.1% of a gallery at 98.05% is not. An earlier version counted a guard as redundant only if it alone covered well under half a percent of the gallery, $R = \sum_j \exp(-\hat{\delta}_j^- / 0.005)$. Under that rule only 2 of the 10 guards that run v7's test solutions could spare (table 4) counted as clearly redundant (weight above 0.75), 2 more were refused by the old ratchet, and v7 learned never to delete. Define

$$\Phi(s) \;=\; -w_{\text{cov}}\,u(s) \;-\; w_{\text{par}}\,k(s) \;-\; w_{\text{prune}}\,R(s), \qquad F(s,s') = \Phi(s') - \Phi(s).$$

F is exactly the dense reward we pay. Its coverage and guard-count parts are $+w_{\text{cov}}\delta^+ - w_{\text{par}}$ on ADD and $-w_{\text{cov}}\delta^- + w_{\text{par}}$ on DEL; its redundancy part $w_{\text{prune}}(R(s) - R(s'))$ is paid on every step. Along any episode,

$$\sum_{t=0}^{T-1} F(s_t, s_{t+1}) \;=\; \Phi(s_T) - \Phi(s_0) \;=\; w_{\text{cov}}\,c_T \;-\; w_{\text{par}}\,k_T \;-\; w_{\text{prune}}\,R(s_T) \;+\; \text{const},$$

where the constant depends only on the start state (0 from an empty gallery). The dense reward therefore depends only on *where the episode ends*, not on the path to it: coverage reached, guards used, and how many of them are free. No sequence of actions can collect more than that endpoint is worth, so no dense term can be farmed. The potentials are differenced with $\gamma = 1$ and the terminal state is not zeroed, so the endpoint value is part of the return rather than cancelled; it is exactly the objective of section 2.5 (cover, with few guards, none of them free). The identity is exact for the undiscounted return. PPO discounts with $\gamma = 0.995$, and episodes are at most 40 steps long, so later terms are weighted by at least $0.995^{40} \approx 0.82$: the shaping is close to, though not exactly, the discounted form $\gamma\Phi(s') - \Phi(s)$ whose policy invariance Ng et al. [14] prove.

Design rule 6 (Anneal iff non-potential)**.** A reward term may keep a fixed weight for the whole of training only if it is a potential difference or part of the objective. Any other term biases the optimum and must reach zero before training ends.

Design rule 6 decides the schedule in fig. 3. The coverage, guard-count and redundancy terms are potentials and keep their weights. The terminal terms are part of the objective. The regularity penalties (w_{proj}, w_{void}, w_{stop}) only discourage actions that are never useful, so they cannot be farmed and change no optimum. The stage-1 scaffolds (frontier, sweep, separation) are the only terms that bias the solution, and they are annealed to zero.

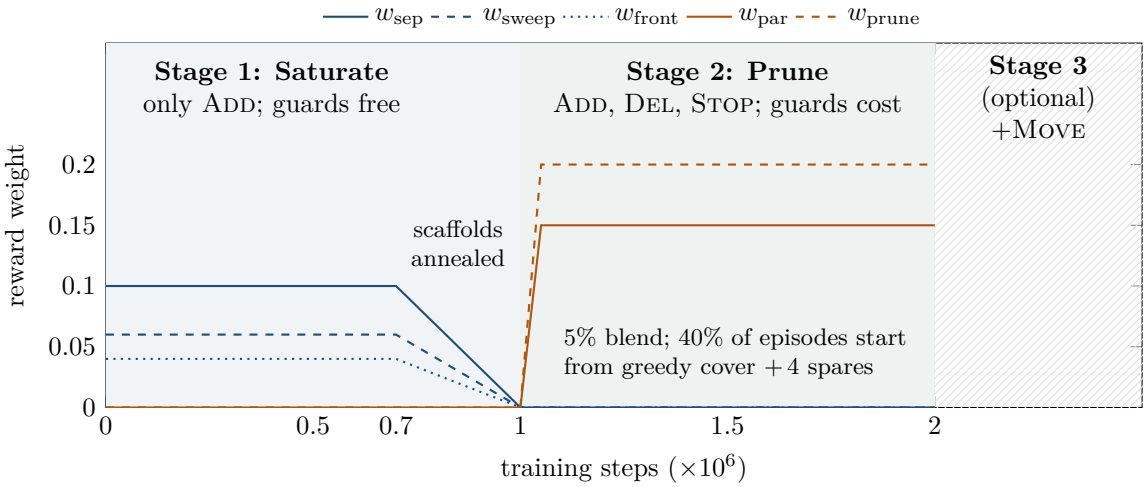


Figure 3: The curriculum and its reward-weight schedule, with the exact values from `rewards.py`. In stage 1 the scaffolds (blue) hold their weight to 70% of the stage and then fall linearly to 0, so the last 30% trains on the unbiased reward (design rule 6). Stage 2 blends from that column to the parsimony weights (orange) over its first 5%, which spares the value function a step change. Not plotted because of scale: the potential term $w_{\text{cov}} = 1$ and the terminal $w_{\text{goal}} = 4$, $w_{\text{goal,min}} = 1$, $w_{\text{fail}} = 2$ are constant; w_{pars} $(0 \to 0.5)$, w_{stop} $(0 \to 2)$ and w_{void} $(0 \to 0.5)$ blend in with w_{par}.

3 Architecture

3.1 What the policy has to do

At every step the policy outputs a mode in $\{\text{Add}, \text{Del}, \text{Stop}\}$ and a point (x, y) in the gallery's bounding box, normalised to $[0, 1]^2$. The geometry of section 2 turns this into five concrete requirements, and each part of the network answers one of them:

R1 Continuous placement. Point guards are continuous (section 2.2), so the action must reach any point, not a fixed candidate set.

R2 Several targets at once. A partly covered hard gallery usually has several separate uncovered pockets. The distribution over points must be able to put mass on all of them, rather than on their average.

R3 Global context. With unlimited range, whether a point is useful depends on what it can see across the whole gallery, not on its neighbourhood.

R4 Fine localisation. The missing 2–4% of a nearly solved gallery is often a thin wedge behind a hole or a reflex corner (fig. 1), a few pixels wide. The network must not blur it away before deciding where to aim.

R5 Unordered modes with masking. ADD, DEL and STOP are unordered alternatives. Stage 1 must disable two of them cleanly, and stage 2 must disable each one in the states where it is never right.

3.2 The network

Figure 4 shows the network (`policy.SpatialPolicy`, about 294k parameters). Its organising idea is to *read the placement off a map* rather than compute coordinates from a summary vector: one branch keeps the image's layout down to a 16×16 grid of cells, and a second branch summarises the whole gallery into one vector that is handed to every cell. The mode and the value of the state, which are properties of the whole gallery, are read from the summary.

A 16×16 map, never flattened (R4). The local branch downsamples the 64×64 image only to the cell grid ($G = 16$, each cell 4×4 pixels) and keeps it there. Every cell has its own feature vector, computed from its own neighbourhood, so a thin uncovered wedge stays a feature of the cells it touches and can raise their scores directly. Aiming at a corner is then no harder than aiming at open floor: it is a different cell of the same map. Flattening the image into one vector before choosing a position would force the network to convert "there is a sliver at the top left" into numeric coordinates, which is exactly the kind of fine spatial detail a coarse summary loses.

Coordinate channels. Convolutions are translation-invariant, but visibility is not: a point next to a wall sees differently from the same local pattern in the middle of the room. Two extra input channels hold each pixel's normalised x and y [12], so every cell knows where it is in the gallery.

A global summary broadcast to every cell (R3). Local features alone cannot tell whether a point would see an uncovered region on the far side of the gallery. The global branch continues to 4×4, flattens, and merges the result with the five scalar stats into $z \in \mathbb{R}^{256}$. A linear map of z to 32 numbers is copied to every cell and concatenated with that cell's local features, so each cell is scored from both its neighbourhood and the state of the whole gallery: what is covered, where the guards are, how much budget is left.

Masking cells outside the gallery. A cell is *live* if any of its 4×4 raster samples lies in the free space (a max-pool of the interior channel). Dead cells, outside P or entirely inside a hole, get logit -10^8 and zero probability (fig. 5). This removes a large and useless part of the search space (in the gallery of fig. 5, 68 of the 256 cells), and the policy never wastes probability on positions that would only be projected back into the gallery and penalised.

Heads and initialisation. The cell head outputs five maps: a score s_k per cell and four Beta parameters, made ≥ 1 by $1 + \mathrm{softplus}(\cdot)$ so that every offset density is unimodal and bounded. In stage 2 a second 1×1 layer on the same hidden map gives DEL its own five maps (section 3.3). The mode and value heads read z, because "stop now" and "how good is this state" are global

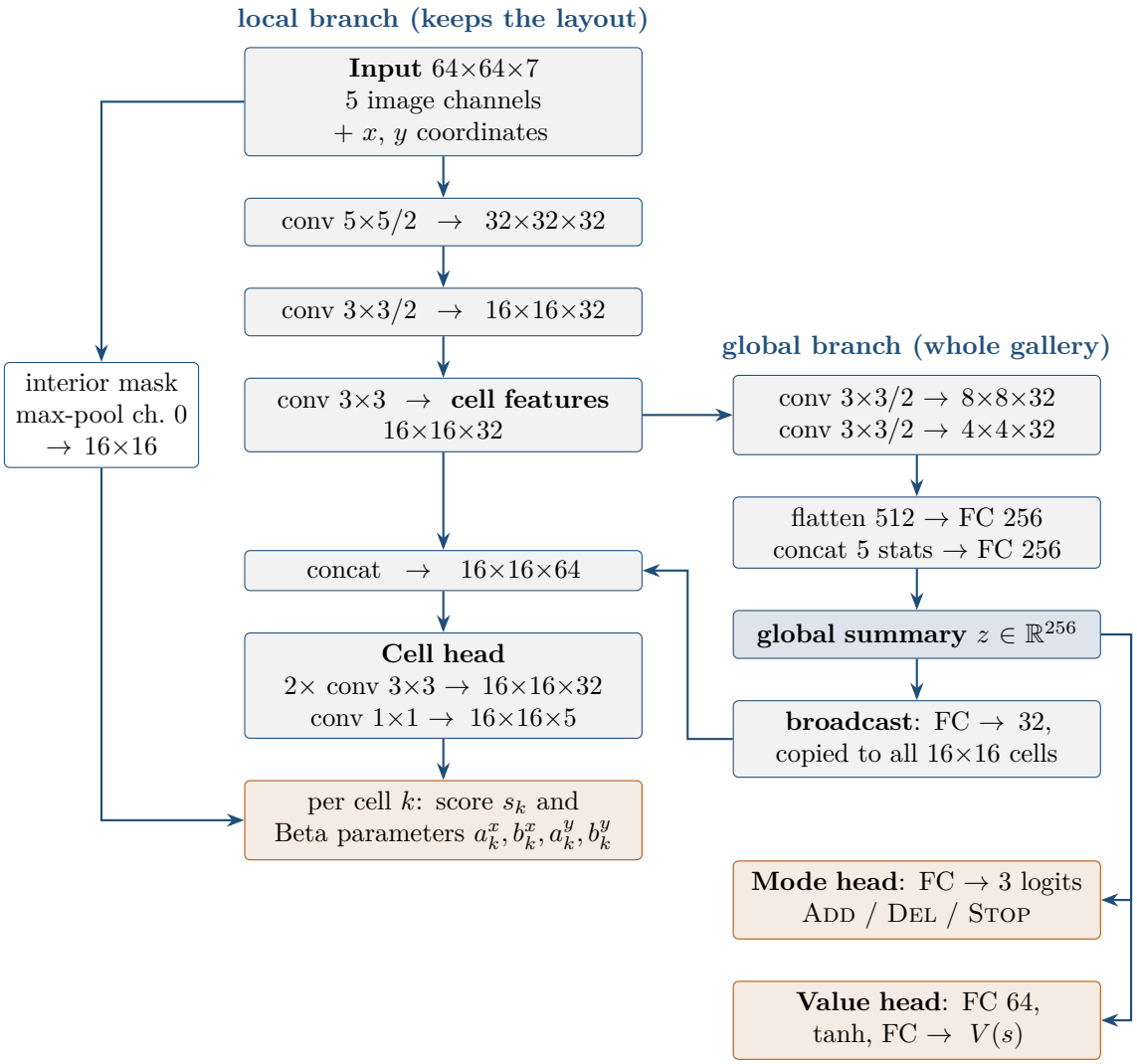


Figure 4: The spatial policy, with tensor sizes. Left: the local branch never flattens the image, so placement is scored per cell on a 16×16 map. Right: the global branch summarises the whole gallery and the scalar stats into z, which is broadcast back to every cell (unlimited-range visibility depends on the whole gallery) and also feeds the mode and value heads. The interior mask removes cells with no free space from the cell distribution. ReLU after every convolution; "/2" is stride 2.

questions. The last layers of the cell and mode heads are initialised orthogonally with gain 0.01, so training starts from near-uniform cells, near-uniform modes and gently centred offsets; Del's layer is initialised the same way when stage 2 adds it.

3.3 The factored action

Index cells by $k = rG + c$ with row r along y and column c along x. The policy samples

$$\mu \sim \mathrm{Cat}(\pi^{\mathrm{mode}}), \qquad k \sim \mathrm{Cat}(\pi^{\mathrm{cell}}_{\mu}), \qquad o_x \sim \mathrm{Beta}(a^x_{\mu,k}, b^x_{\mu,k}), \quad o_y \sim \mathrm{Beta}(a^y_{\mu,k}, b^y_{\mu,k}),$$

$$(x, y) \;=\; \Big(\frac{c + o_x}{G},\; \frac{r + o_y}{G}\Big), \qquad \pi^{\mathrm{cell}}_{\mu,k} \propto \exp(s_{\mu,k})\, \mathbf{1}[\text{cell } k \text{ is live for } \mu]\,,$$

and sends the vector $[\mu, x, y]$ to the environment (fig. 5). Add and Del each have their own maps; Stop ignores the point and uses Add's. A cell is live for Add if it holds gallery interior, and for Del if it also holds a guard: Del removes the guard nearest its point, so no target is lost.

Why Del has its own map. Up to v10 one map served every mode, and the mode and the cell were drawn independently, so where the policy pointed could not depend on what it was about to do. ADD is about two thirds of all actions and learns to point at uncovered space; the guards worth deleting sit where coverage overlaps. A DEL therefore pointed roughly where an ADD would go: on hard galleries most were refused, a median quarter to a third of the diagonal away from any free guard (section 3.6). Stage 1 never deletes and has only ADD's map; stage 2 adds DEL's, starting near-uniform over the guards' cells rather than as a copy of ADD's.

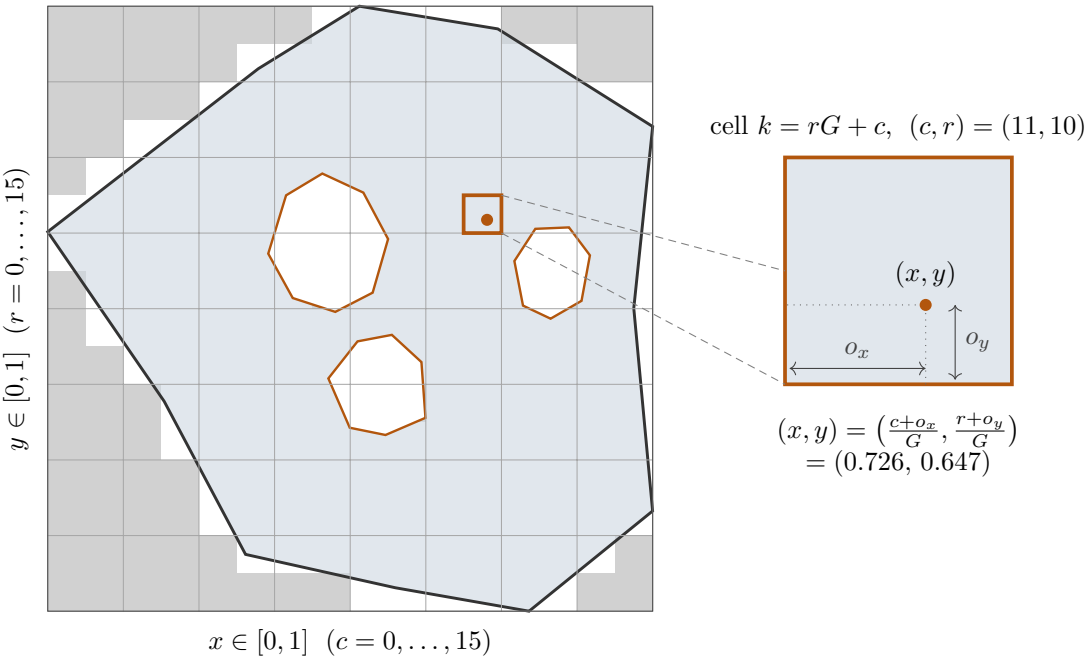


Figure 5: A region is chosen discretely, the point inside it continuously. Left: the 16×16 cell grid over the bounding box of a real gallery (mid test gallery #30). Grey cells contain no free space and are masked. The policy samples a live cell (orange, $(c, r) = (11, 10)$), then a continuous offset from that cell's two Beta distributions, $o_x \sim \text{Beta}(a_k^x, b_k^x)$ and $o_y \sim \text{Beta}(a_k^y, b_k^y)$. Right: the offset fixes the exact point, here $o_x = 0.62$, $o_y = 0.35$. Any point in the gallery can be produced, so the grid organises the choice without restricting where a guard can go.

Why a distribution over regions (R2). When several parts of the gallery are still uncovered, the policy should be able to aim at any of them. A categorical over cells assigns independent probability to every region, so it can put mass on two opposite corners at once. A single Gaussian over (x, y), or any method that outputs one averaged point, cannot: its mean falls between the targets, usually in space that is already covered (fig. 6). The same holds for the deterministic action used in evaluation: the most probable cell is an actual choice of one region, while the mean of a unimodal distribution is a compromise between regions.

Why a continuous offset inside the cell (R1). The Beta offset makes the action continuous while keeping the density exact.

Proposition 7 (The action is continuous)**.** *For $\mu \neq$ STOP, the induced distribution of (x, y) has density*

$$p(x, y) \;=\; G^2 \, \pi^{\text{cell}}_{k(x,y)} \, \text{Beta}\big(o_x(x)\big) \, \text{Beta}\big(o_y(y)\big)$$

on $[0, 1]^2$, where $k(x, y) = \lfloor Gy \rfloor G + \lfloor Gx \rfloor$, $o_x(x) = Gx - \lfloor Gx \rfloor$ and $o_y(y) = Gy - \lfloor Gy \rfloor$. It is positive on every live cell.

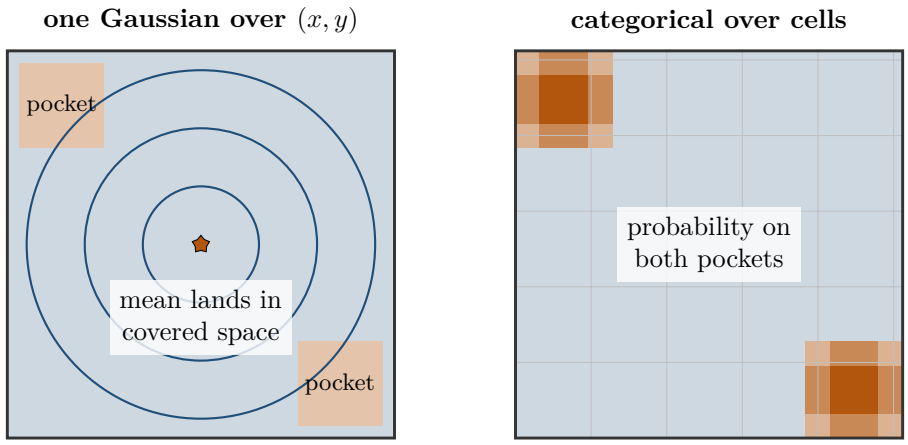


Figure 6: Why the policy chooses a region instead of one averaged point. Two uncovered pockets (orange) sit in opposite corners of an otherwise covered gallery (schematic). *Left:* a single Gaussian that tries to serve both pockets centres between them, so its mean, the deterministic action, lands in already-covered space, and its samples mostly do too. *Right:* the categorical over cells puts high probability (darker cells) on both pockets independently, so every sample aims at real work, and the most probable cell is one of the pockets.

Proof. Cells partition $[0, 1)^2$, so each point belongs to exactly one cell and determines its offsets uniquely. Within a cell the map from offsets to (x, y) scales each coordinate by $1/G$, so the Jacobian contributes G^2. Beta densities with parameters ≥ 1 are positive on $(0, 1)$. □

The grid organises *how* the policy decides; it does not restrict *where* a guard can go. This is the difference from candidate-set solvers [5, 15] and from a plain discrete "spatial action map" [17], which would snap every guard to a cell centre and give up advantage (1) of section 2.2. The Beta distribution is the natural choice for a bounded offset: it lives on $(0, 1)$ exactly, so no probability is clipped at the cell edges, which avoids the boundary bias of clipped Gaussians [3].

Log-probability, entropy, deterministic action. Given a stored action $[\mu, x, y]$, the code recovers k, o_x, o_y and computes

$$\log \pi(a \mid s) \;=\; \log \pi_{\mu}^{\text{mode}} \;+\; \mathbf{1}[\mu \neq \textsc{Stop}] \left(\log \pi_{\mu,k}^{\text{cell}} + \log \text{Beta}_{\mu,k}(o_x) + \log \text{Beta}_{\mu,k}(o_y)\right).$$

The constant $\log G^2$ is dropped because it cancels in PPO's probability ratio. The point term is excluded for Stop because a Stop ignores the point, so including it would only add gradient noise to a decision that does not depend on it. Offsets are clamped to $[10^{-5}, 1 - 10^{-5}]$, which keeps sampled points strictly inside their cell and the Beta log-densities finite. The entropy bonus is that of the joint choice,

$$\mathcal{H} \;=\; \mathcal{H}(\pi^{\text{mode}}) + \textstyle\sum_{\mu \in \{\textsc{Add},\textsc{Del}\}} \pi_{\mu}^{\text{mode}}\, \mathcal{H}_{\mu},$$

$$\mathcal{H}_{\mu} = \mathcal{H}(\pi_{\mu}^{\text{cell}}) + \textstyle\sum_{k} \pi_{\mu,k}^{\text{cell}} \big(\mathcal{H}(\text{Beta}_{\mu,k}^{x}) + \mathcal{H}(\text{Beta}_{\mu,k}^{y})\big),$$

with the weights π_{μ}^{mode} held constant in the gradient, so the bonus cannot push the mode towards whichever map is more spread out (Add's covers the gallery, Del's only the guards). In stage 1 $\pi_{\textsc{Add}}^{\text{mode}} = 1$ and this is the true joint entropy. The deterministic action used for evaluation is the most probable mode, the most probable cell of that mode's map, and the Beta means $a/(a+b)$ inside it.

3.4 The mode head across the stage switch (R5)

A categorical, not a number. The three modes are separate logits. Moving probability from ADD to STOP does not pass through DEL, as it would if the mode were one continuous output cut into bands: learning "stop once covered" must not require first becoming worse at something else.

Stage 1: masking at the logits. The DEL and STOP logits are replaced by -10^8, so their probability is exactly zero and they contribute nothing to the log-probability or the entropy [9]. The environment also decodes every stage-1 action as ADD, so the MDP is correct for any policy.

Stage 2: priming. Masked logits receive no gradient, so at the switch the mode head is still at its initialisation and would choose among the three modes arbitrarily. Instead, we zero the mode head's weights and set its biases so that $\pi^{\text{mode}} = (0.8, 0.1, 0.1)$ in every state (`--add-prob 0.8`). Stage 2 therefore starts by behaving as stage 1 ended, mostly adding, while sampling DEL and STOP from the first update. The placement heads are left as stage 1 trained them. Priming only sets the starting point; keeping DEL explored afterwards is the job of the mode's own entropy bonus (section 3.5).

Stage 2: three masks on moves that are never right. Some mode choices are wrong in every state of a given kind, and the observation already says which states those are. Learning that from reward costs samples and, as the development runs showed (table 4), sometimes fails outright. The policy therefore masks them at the logits, as in stage 1, reading the two scalars that decide them (table 6):

Stop below $c^\dagger$ (`stop_below`). Stopping short is never the better answer. Left to learn it, a policy kept stopping at 96–98% on hard galleries.

Stop while a guard is free (`stop_while_free`). Guards cost reward, so a free guard left at the end is always worse than deleting it. Left to learn it, v7 stopped with every spare guard still in place.

Del below $c^\dagger$ (`delete_below`). Below the threshold a deletion can only lose more coverage, and deleting a guard to re-add it elsewhere nets nothing, so ADD is the only useful move. Left to learn it, v8 kept choosing DEL at 97% until two refusals ended the episode uncovered.

None of these takes a choice away from a state where it could help. A DEL made at or above $c^\dagger$ may still dip coverage below it; the masks then leave ADD as the only move, which forces the repair. ADD is never masked, so at least one mode is always available.

Stage 2: Add kept out of cells known to add nothing. A fourth mask acts on ADD's cell map rather than on the mode (`skip_redundant_cells`). A guard that sees nothing any other guard does not see (raster $\hat{\delta}_j^- = 0$) is drawn at 255 in channel 4 at any coverage, and ADD may not choose its cell. An ADD that gains nothing leaves the observation almost unchanged, so a deterministic policy repeats it: v11 stacked eleven guards on one spot of a hard gallery at 97.6% until the budget ended the episode (section 3.6). After one such ADD the new guard and the one it duplicates both have $\hat{\delta}_j^- = 0$, and the cell closes. Unlike the mode masks this one is a heuristic: a different point of the same cell could still see something new, and a point aimed into a hole is projected into the gallery, possibly into a cell the mask does not cover.

3.5 PPO configuration

Table 2 lists the settings. $\gamma = 0.995$ is close to 1 because episodes are short (at most 40 steps) and the terminal bonus must reach the first placement. We do not stop updates early on a

Table 2: PPO settings (`train.py`). One model runs both stages; the step counter continues across the switch.

learning rate	3×10^{-4} (stage 2: 10^{-4})	discount γ	0.995
environments	16 (subprocesses)	GAE λ	0.95
rollout length	256 per env (4096 per update)	clip range	0.2
minibatch	512	epochs per update	10 (stage 2: 5)
entropy coefficient	0.005 (stage-2 mode: 0.03)	value coefficient	0.5
gradient clip	0.5	target KL	off
steps	10^6 per stage	device	Apple MPS

KL threshold: a cap of 0.03 ended updates after so few gradient steps that the placement head barely trained.

Smaller steps in stage 2. Stage 2 changes the reward under a policy that already covers every tier. Its job is to add DEL and STOP on top of that skill, not to relearn placement, so it uses a third of stage 1's learning rate and half the epochs per update. At stage 1's step size, single updates reached a KL divergence above 1 late in stage 2, and the placement skill on hard galleries degraded within 150k steps (solved fell from about 95% to 50% in training).

Separate entropy bonuses for the mode and for placement. The bonus acts on the joint entropy of section 3.3, and that is dominated by the cell: up to $\ln 256 \approx 5.5$ nats against $\ln 3 \approx 1.1$ for the mode. One coefficient cannot serve both. Sized for placement (0.005), it barely acts on the mode: priming starts DEL at 10%, but a correct deletion earns only about +0.35 while a mis-aimed one is refused or costs coverage, and under a noisy critic the mode head converged on stopping at once, keeping every redundant guard. Raised enough to keep DEL in play, it mainly flattens the cell distribution, making placement less precise where hard galleries need precise corner guards. Stage 2 therefore weights the mode term separately,

$$\mathcal{H}_w = 6\,\mathcal{H}(\pi^{\text{mode}}) + \textstyle\sum_{\mu \in \{\text{ADD},\text{DEL}\}} \pi_\mu^{\text{mode}}\,\mathcal{H}_\mu,$$

so with coefficient 0.005 the mode receives an effective bonus of 0.03 and placement keeps 0.005. The share of ADD, DEL and STOP among actions is logged per rollout, so a mode head that abandons DEL is visible long before the guard counts show it.

Model selection. Even with smaller steps, the last stage-2 policy need not be the best one. Every 100k steps we save a checkpoint and score it on a separate validation set (60 galleries, 20 per tier, generated from their own seed; table 5), running each gallery once with deterministic actions. A gallery scores

$$q = \begin{cases} k_{\text{ref}}/k & \text{if solved,} \\ 0 & \text{otherwise,} \end{cases}$$

so $q = 1$ matches greedy and $q > 1$ beats it; the checkpoint's score is the mean of q, and the highest-scoring checkpoint is the result. The starting policy is scored too, so the kept model is never worse on validation than where stage 2 began. One number is needed because ranking by solved rate first fails: a policy that never stops fills the 16-guard budget, covers everything, and would outrank one that solves slightly fewer galleries with a third of the guards. No checkpoint is chosen by its test score, though the test set was also used to diagnose the development runs (section 3.6).

Hardware. Training runs on Apple's MPS GPU: a forward and backward pass over 512 samples takes 12 ms there versus 600 ms on the CPU. Beta sampling, the one operation MPS

lacks, is done on the CPU for the single offset drawn per environment per step. $G = 16$ is a compromise: 256 regions separate distinct pockets while keeping the categorical small enough to explore, and $G = 32$ would double the network's cost without changing what positions are reachable.

3.6 Results

Stage 2 cuts guards by about 12% **and leaves no free guard.** The current implementation, trained as run v11 (`runs/agp_v11`, stage 2 continuing from the stage-1 checkpoint of `runs/agp_v2`), solves 89 of the 90 test galleries at 1.191 times the greedy guard count, against 1.347 for stage 1 and 1.196 for v7, the best earlier run. None of its solutions holds a guard that could be deleted, and from a pre-filled start it removes 4.0, 4.0 and 3.6 of the four spares on easy, mid and hard galleries. The one failure is a deterministic loop: on a hard gallery it adds guards at the same spot until the budget runs out.

Table 3 gives the standard evaluation (`agp.train eval`: deterministic actions, 90 test galleries, starting from zero guards) for stage 1, for v7, the stage-2 run with the lowest guard count before v11, for v10, and for v11, the current implementation. v7 ran under an earlier rule set: no free-guard rule, no STOP mask for free guards and no DEL mask (table 4). Stage 1 is scored with v9's observation code; its training saw the older channel 4, which the add-only policy barely uses.

Table 3: Test-set evaluation from zero guards, deterministic actions. k/k_{ref} is mean k over mean k_{ref} on the solved galleries, as `agp.train eval` prints it.

model	tier	solved	mean coverage	mean k (solved)	k/k_{ref} (solved)
stage 1 (`agp_v2`)	easy	100%	0.996	2.63	1.362
	mid	100%	0.994	3.77	1.413
	hard	100%	0.988	7.83	1.313
	all	100%	0.993	4.74	1.347
stage 2, v7	easy	100%	0.996	2.27	1.172
	mid	100%	0.991	3.17	1.188
	hard	100%	0.988	7.20	1.207
	all	100%	0.991	4.21	1.196
stage 2, v10	easy	100%	0.994	2.20	1.138
	mid	96.7%	0.992	3.24	1.205
	hard	100%	0.987	7.33	1.229
	all	98.9%	0.991	4.27	1.206
stage 2, v11 (current)	easy	100%	0.995	2.30	1.190
	mid	100%	0.992	3.03	1.137
	hard	96.7%	0.985	7.38	1.216
	all	98.9%	0.991	4.20	1.191

Stage 2 mostly improves placement. v7 uses 11% fewer guards than stage 1 (8–16% per tier) without deleting a single guard. Pruning cannot explain that gap: removing every free guard from stage 1's solutions takes it only from 1.384 to 1.295 (mean per-gallery ratio), while v7 reaches 1.213. Most of stage 1's extra guards are each needed, just badly placed, and only placing differently from the start removes them. This is the case for starting stage-2 episodes from zero guards rather than from stage 1's finished cover.

The development runs. Table 4 lists each stage-2 run and the change that led to the next one. Every run starts from the same stage-1 checkpoint. Each change answers a specific failure, and the three policy masks of section 3 come from v6, v7 and v8.

Table 4: Stage-2 development runs on the test set, deterministic, from zero guards. k/k_{ref} is the mean per-gallery ratio over solved galleries (it differs slightly from table 3). "Deletions" are accepted deletions per gallery; "free left" is the number of free guards still in the final solution on hard galleries.

run	change from the previous run	solved	k/k_{ref}	deletions	free left
stage 1	add only; guards free	100%	1.384	–	0.37
v4	stage 2 as in section 3.5; terminal bonus floored at w_{goal}, STOP ends the episode	96.7%	1.356	0	0.70
v5	terminal bonus uncapped below (floor $w_{\text{goal,min}}$); STOP below $c^{\dagger}$ refused; step limit ends the episode	82.2%	1.240	0.01	0.48
v6	w_{void} 0.1 → 0.5; two refused actions in a row end the episode	95.6%	1.302	0	0.77
v7	STOP masked below $c^{\dagger}$	100%	1.213	0	0.13
v8	free means the $c^{\dagger}$ rule; ratchet floor $\min(c^{\text{best}} - 0.01, c^{\dagger})$; STOP masked while a guard is free	75.6%	1.287	0.03	0.00
v8 + mask	DEL masked below $c^{\dagger}$, at evaluation only	100%	1.319	0.09	0.20
v9	v8 trained with the DEL mask	100%	1.232	0.10	0.40
v10	channel 4 redrawn: free guards brightest, never 0	98.9%	1.215	0.22	0.33
v11	DEL gets its own cell map, masked to the guards' cells (current)	98.9%	1.201	0.42	0.00

What the runs show:

- **v4–v6:** none of them deleted one of its own guards. v4 never deleted at all; v5 and v6 removed about half of the spares on pre-filled easy and mid galleries. v5's deterministic policy also looped on refused actions until the step limit, which v6's larger w_{void} and the two-refusal ending removed. v6's remaining failures were all STOP at 96–98% on hard galleries.
- **v7:** with STOP masked below $c^{\dagger}$ every gallery was solved and placement improved steadily, but DEL fell from 10% of actions to 0 within 50k steps. Under the old redundancy rule most deletable guards looked load-bearing (section 2.6), and stopping was always safe.
- **v8:** with the free-guard rule DEL stayed at about 40% of actions, and on pre-filled easy galleries it removed 3.6 of the 4 spares. But DEL took over the nearly-covered states too: the failures added guards to just under 98%, then chose DEL until two refusals ended the episode uncovered. The DEL mask alone, with no retraining, solves all 90 (the v8 rows are its checkpoint at 1.1M steps, 10% into stage 2).
- **v9:** trained with the DEL mask, it solved every training episode throughout and DEL held at about 35% of actions to the end. Its placement improved more slowly than v7's for most of the stage (validation ratio 1.29 at 1.5M steps against v7's 1.22) and caught up only in the last quarter (1.226 against 1.205 at the end, still falling). Deletion aim did not improve on hard galleries: from a pre-filled start (greedy plus four spares) it removes about 2.9, 2.6 and 0.9 guards on easy, mid and hard galleries and leaves 1.0, 1.3 and 2.8 free guards, and 8 of the 30 hard solutions from zero guards still hold a free guard. Removing those would take v9 from 1.232 to 1.195 (mean per-gallery ratio). The model saved is the final one: the best-on-validation checkpoint (2.0M steps) has a lower ratio (1.223) but fails one hard gallery.
- **v10:** channel 4 used to draw the cheapest free guards at 0, like empty floor, and v9's refused deletions pointed a median quarter to a third of the diagonal away from any free guard. With free guards drawn brightest, placement improved much faster (validation ratio 1.220 at 1.5M steps against v9's 1.291) and settled between 1.19 and 1.23; whether the redrawing or run-to-run variation caused that, one run cannot say. On the test set v10

is level with v7 (1.215 against 1.213 deterministic, 1.219 against 1.223 sampled) and fails one mid gallery. Deletion aim on hard galleries did not improve: from a pre-filled start it removes 3.3, 2.6 and 0.5 spares on easy, mid and hard galleries, and refused deletions still point far from any free guard. The likely cause is in the network rather than the image: the mode and the cell are drawn independently from one shared cell map, so a DEL points with the map that ADD, two thirds of all actions, has shaped towards uncovered space, while free guards sit where coverage is overlapping.

- **v11:** with its own cell map (section 3.3) DEL aims at guards. From a pre-filled start it removes 4.0, 4.0 and 3.6 of the four spares on easy, mid and hard galleries, against v10's 3.3, 2.6 and 0.5; on hard galleries 108 of its 113 deletions hit a free guard, against v10's 13 of 73. From zero guards no solution on any tier keeps a free guard, so pruning its output changes nothing. Placement improved as well: the validation ratio reached 1.167 (v10: 1.194, v7: 1.205), and the test ratio is the lowest of any run (1.191; 1.201 per gallery, 1.197 sampled, 1.122 best of five). The one deterministic failure is gallery 29 of the hard tier: after five guards at 97.2% every further ADD lands where it adds nothing, until the budget of 16 ends the episode; sampled actions solve it. With the ADD cell mask (section 3) added at evaluation, without retraining, gallery 29 is solved with four guards, but hard gallery 17 then fails another way: it places guards in new cells, each adding almost nothing, and stalls at 97.85% when the budget runs out. The ratio is unchanged (1.200 per gallery).

A caveat on the test set. The development runs were diagnosed on the test galleries, and several changes answer failures seen there. The test numbers are therefore not fully held out; a claim about generalisation should be checked on a fresh split generated from a new seed.

4 MDP formulation and reward

Figure 7 summarises one step of the decision process. The rest of this section states each part precisely.

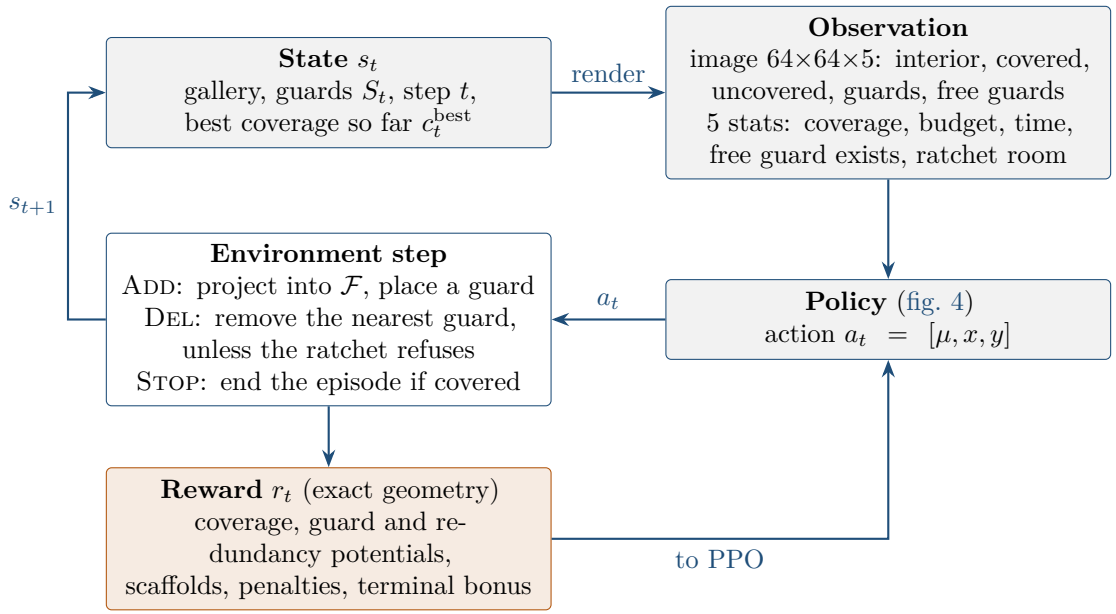


Figure 7: One step of the MDP. The state is rendered into a 5-channel image and five scalars; the policy outputs a mode and a point; the environment applies it with exact geometry and returns the next state and a reward. Transitions are deterministic; the only randomness is the gallery drawn at reset and the policy's own sampling.

4.1 Instances

Each episode draws a gallery uniformly from the training set and applies a random symmetry of the square (one of 8 rotations and reflections about the gallery centre), which also transforms the stored greedy guards. Augmentation is switched off for evaluation. The generator (`polygons.py`) produces three tiers (table 5).

Table 5: Difficulty tiers. 200 training, 20 validation and 30 test galleries per tier, each split from its own seed.

tier	outer vertices	holes	vertices per hole
easy	8–12	1–3	4–6
mid	10–16	2–5	4–8
hard	16–26	5–9	5–10

4.2 State, actions, termination

The state is $s_t = (G, S_t, t, c_t^{\text{best}})$: the gallery, the multiset of placed guards, the step count, and the best coverage reached so far in the episode. The action $[\mu, x, y]$ is decoded to the point $q = (x_{\min} + x\,w,\ y_{\min} + y\,h)$.

Add. If $q \notin \operatorname{int}\mathcal{F}$ (outside P or inside a hole), it is projected to the nearest boundary point and then moved towards a fixed interior reference point by the first $t \in \{0.01, 0.05, 0.10, 0.25, 0.50\}$ that lands strictly inside. A guard is *always* placed: rejecting the action instead would leave the state unchanged, and a deterministic policy that emits the same invalid point would repeat it forever. The projection is penalised (section 4.4) so the policy still learns the shape.

Del. Removes the guard nearest to q. Addressing a guard by a point rather than an index keeps the action fixed-size and independent of guard order, and reuses the same point head as Add. Deletions are subject to a *coverage ratchet*: a deletion is refused (state unchanged, penalised) if it would bring coverage below

$$\min(c_t^{\text{best}} - 0.01,\ c^\dagger).$$

Any deletion that keeps the gallery solved therefore goes through, however much coverage it costs; a dip below $c^\dagger$ is allowed only within 1% of the best coverage so far, and is then repaired with Add. Undoing real progress is impossible rather than merely costly. The earlier floor, $c_t^{\text{best}} - 0.01$ alone, refused 2 of the 10 deletions that would have tidied v7's test solutions.

Stop. Ends the episode with the current guard set if $f(S_t) \geq c^\dagger$. Below the threshold it is refused: the state is unchanged and the step is penalised. A deletion that dips under $c^\dagger$ is then something to repair, not a way to end the episode. (The policy masks Stop there anyway, section 3; the refusal keeps the MDP correct for any policy.)

Termination. The episode *terminates*, with the terminal reward of section 4.4, on an accepted Stop; when an Add reaches the budget of 16 guards; in stage 1 only, as soon as $f \geq c^\dagger$ (in stage 2 it continues so that pruning can happen); after 40 steps; and after two refused actions in a row (a refused Del or Stop). There is no truncation. The step count is in the observation, and with Stop refused below $c^\dagger$, bootstrapping at the step limit would let a policy that cannot cover avoid the failure penalty by stalling. A refused action leaves the state unchanged, so a deterministic policy that takes one will take it again forever; the two-refusal limit ends that loop, and the episode is scored like any other ending, covered or not.

Why a budget of 16. Greedy needs at most 11 guards on any training gallery, so 16 leaves room for stage 1 to over-fill and stage 2 to prune. It is tight enough that guard placement still matters, and the observation reports the fraction of budget used. The reward sizes in section 4.6 are checked against this value and must be rechecked if it changes.

4.3 Observation

Table 6: Observation (`env.py`). Image channels are 0/255 masks except channel 4.

Image channels (64×64 over the bounding box)	
0 interior	pixels inside $\mathcal{F}$
1 covered	pixels seen by at least one guard
2 uncovered	interior pixels seen by no guard: the work left
3 guards	each guard as a 3×3 blob, so it survives strided convolutions
4 free guards	each guard's blob: a free guard at 255 down to 128, brightest when its deletion costs nothing and dimmer by the share of the headroom $f(S_t) - c^\dagger$ it would use up; a load-bearing guard at 64. Bright means "delete this", and no guard is drawn at 0, the value of empty floor. Below $c^\dagger$ no guard is free, but one that sees nothing no other guard sees is still drawn at 255. The gap between the bands keeps the two apart however close to the line a guard is
Scalars (all clipped to $[0, 1]$)	
coverage	exact $f(S_t)$
budget used	$k_t/16$
time used	$t/40$
free guard exists	0 if some guard is free, else 1; the policy masks STOP on it
ratchet room	$f(S_t) - \min(c_t^{\text{best}} - 0.01, c^\dagger)$: coverage a deletion may still cost

Table 6 lists the observation. Channel 2 tells the policy where the work is; channel 4 is what makes DEL learnable at all, since without it a load-bearing guard and a free one look identical. Free guards are drawn brighter than load-bearing ones and never at 0: up to v9 the order was reversed, the cheapest free guards were drawn at 0 like empty floor, and v9's refused deletions pointed a median quarter to a third of the diagonal away from any free guard. Which guards are free is decided on the exact δ^-, so it agrees with what a deletion would actually do. The exact value is a polygon union over the other guards, about 3 ms each, and computing it for every guard on every step made training 2–5× slower. The raster estimate $\hat{\delta}_j^-$, pixels seen by guard j and by no other guard, is off by under 1% of the gallery, so a guard whose estimate is more than 1% from the headroom is settled by the raster alone, and only the few near the line get the exact union. On 3,385 guards from random play this matched the exact rule every time.

4.4 Reward

Let g be the placed (projected) point, $\iota = \mathbf{1}[q \notin \operatorname{int} \mathcal{F}]$, D the bounding-box diagonal and $j^\star$ the guard nearest q. The per-step reward is

$$r_t \;=\; r_t^{\mu} \;+\; w_{\text{prune}}\big(R(s_t) - R(s_{t+1})\big),$$

$$r_t^{\mu} \;=\; \begin{cases} w_{\text{cov}}\,\delta^+ - w_{\text{par}} - w_{\text{proj}}\,\iota + r_t^{\text{scaf}}, & \mu = \text{ADD}, \\ -w_{\text{cov}}\,\delta_{j^\star}^- + w_{\text{par}}, & \mu = \text{DEL, accepted}, \\ -w_{\text{void}}, & \mu = \text{DEL, refused or no guards}, \\ -w_{\text{stop}}\mathbf{1}\left[f(S_t) < c^\dagger\right], & \mu = \text{STOP}, \end{cases}$$

where R is the number of free guards of section 2.6, recomputed after every step whatever the mode, plus, on termination only,

$$r^{\text{term}} = \begin{cases} \max(w_{\text{goal}} + w_{\text{pars}}(k_{\text{ref}} - k_T),\ w_{\text{goal,min}}), & f(S_T) \geq c^\dagger, \\ -w_{\text{fail}}, & \text{otherwise.} \end{cases}$$

Every guard below k_{ref} earns w_{pars} and every guard above it costs w_{pars}, so spare guards left at the end are visibly worse than clearing them; the floor keeps even a wasteful cover (+1) better than failing (−2). An earlier version floored the parsimony part at zero, $w_{\text{pars}} \max(k_{\text{ref}} - k_T, 0)$, so a policy at or above greedy's count lost nothing by stopping with spares. The scaffold reward of a placement is

$$r_t^{\text{scaf}} = w_{\text{front}} \mathbf{1}[g \notin \mathcal{C}(S_t)] + w_{\text{sweep}}\, \kappa_t(g) - w_{\text{sep}} \mathbf{1}[\min_j \|g - g_j\| < 0.15\, D]\,.$$

The sweep kernel favours the wall early and the interior later: $\kappa_t(g) = \exp(-(d_\partial(g)/D - m_t)^2/(2 \cdot 0.05^2))$ with $m_t = 0.02 + 0.23\, k_t/16$. Table 7 gives every weight; fig. 3 shows how they move over training.

Optional: scaling parsimony by shape complexity. With `--scale-by-reflex`, the parsimony weights w_{par}, w_{prune} and w_{pars} of an episode are multiplied by $\text{clip}(r_G/\bar{r},\, 0.5,\, 2)$, where r_G is the gallery's number of reflex vertices (corners with an interior angle above 180°; a corner of a hole is reflex for the gallery when it is convex for the hole) and $\bar{r}$ its mean over the training set. The reflex count is the usual complexity measure for the problem: in a simple polygon, r reflex vertices never need more than r guards. It correlates 0.92 with k_{ref} without using the tier label: averages are 10.6, 23.9 and 55.2 on the easy, mid and hard training galleries, with overlapping ranges. Guards then cost more, and deletions pay more, on complex shapes. A constant factor on a potential is still a potential, so the scaling cannot be farmed. It is off in every run reported here.

Table 7: Reward terms and their weights (`rewards.py`). "Potential" marks the terms that are differences of the potential in section 2.6.

group	weight	role	SATURATE	PRUNE
coverage	w_{cov}	area gained / lost (potential)	1.0	1.0
	w_{goal}	terminal bonus for $f \geq c^\dagger$	4.0	4.0
	$w_{\text{goal,min}}$	floor on that bonus after the parsimony charge	1.0	1.0
	w_{fail}	terminal penalty otherwise	2.0	2.0
parsimony	w_{par}	cost per guard, refunded on deletion (potential)	0	0.15
	w_{prune}	drop in the number of free guards R (potential)	0	0.20
	w_{pars}	terminal bonus per guard below k_{ref}, charge per guard above	0	0.50
scaffolds	w_{front}	guard placed in uncovered space	0.04	0
	w_{sweep}	boundary-first, then inward	0.06	0
	w_{sep}	penalty for a guard within $0.15D$ of another	0.10	0
regularity	w_{proj}	aimed outside $\mathcal{F}$	0.25	0.25
	w_{void}	impossible or refused deletion	0	0.50
	w_{stop}	STOP before $c^\dagger$ (refused)	0	2.0

Why each group exists. The coverage, guard-count and redundancy potentials carry the objective (section 2.6); the redundancy potential is what pays for deleting free guards. The terminal terms make crossing $c^\dagger$ the thing that matters, and w_{pars} scores the final count against greedy in both directions. The scaffolds make early coverage learning fast: aim at uncovered space, trace the boundary first, do not stack guards; being non-potential, they are removed

before stage 1 ends. The regularity terms keep actions meaningful: aim inside the gallery, do not request impossible deletions, do not stop short.

4.5 Schedule

- **Stage 1.** The scaffolds hold full weight for the first 70% of the stage and then fall linearly to 0 at its end (fig. 3).
- **Stage 2.** The weights blend linearly from the stage-1 column (scaffolds at zero) to the stage-2 column over the first 5% of the stage, so the value function is not hit by a step change in its targets. 40% of stage-2 episodes start *presaturated*: the stored greedy cover (before greedy's own prune step) plus 4 guards at random interior points. The gallery is covered before the extras go in, so the start holds about 4 deletable guards and DEL gets a learning signal from the first step. Almost all of them come from the extras: greedy's prune removes a guard in only 59 of 600 training galleries.
- A callback pushes the current weights to every environment every 4096 steps; the environment has no notion of stage beyond its weights and the `allow_delete` flag.

4.6 Weight calibration

The weights are sized so that no shortcut beats solving the gallery.

- **No scaffold farming (stage 1).** $w_{\text{front}} + w_{\text{sweep}} \leq 0.10$ per placement, so a full budget of 16 earns at most 1.6 in scaffolds, well below the 4 for reaching $c^\dagger$ plus the 2 for avoiding failure.
- **Parsimony never makes failure attractive (stage 2).** A full budget costs $16 \times 0.15 = 2.4$, less than the $w_{\text{goal}} + w_{\text{fail}} = 6$ swing between solving and failing.
- **Pruning is worth doing.** Deleting a free guard lowers R by at least 1, so it returns at least $w_{\text{par}} + w_{\text{prune}} = 0.35$ at once, less the coverage it costs (at most the headroom above $c^\dagger$, so under 0.02), plus $w_{\text{pars}} = 0.5$ at the end: about 0.85 in all. It returns more when the deletion also makes a partner load-bearing: if two guards cover the same region, both are free, and removing one drops R by 2, for $0.15 + 0.20 \times 2 = 0.55$ at once.
- **A refused deletion costs more than a lucky one gains, often enough.** A refused DEL costs $w_{\text{void}} = 0.5$, so aiming DEL pays only where it hits a free guard more than about $0.5/(0.5+0.85) \approx 37\%$ of the time. At 0.1 it paid even at one hit in eight, and DEL stayed the likeliest action in states where every guard was load-bearing, which a deterministic policy then repeated until the step limit.
- **Moving a guard is free.** A DEL followed by an ADD nets out in every potential, so relocating a guard costs nothing in itself and pays off only through the final count.
- **Stopping early is costly.** A refused STOP costs $w_{\text{stop}} = 2$ and the episode goes on; the policy masks it in any case.

Why the pruning reward is a potential. The obvious alternative, a bonus w_{prune} paid on each deletion of a free guard, can be farmed. Adding a guard in already covered space costs $w_{\text{par}} = 0.15$, and deleting it straight away pays $w_{\text{par}} + w_{\text{prune}} = 0.35$: a net $+0.20$ per two steps, which over a 40-step episode approaches the goal bonus. Paying the change of R instead closes the loop exactly: adding a guard in covered space raises R by 1 and costs w_{prune}, deleting it lowers R by 1 and pays w_{prune} back, and the cycle nets zero. Over any episode the pruning reward totals $w_{\text{prune}}(R(s_0) - R(s_T))$, which is bounded and is earned only by ending with fewer free guards than at the start.

5 Open settings and a possible relocation move

5.1 Settings that were chosen, not tuned

Two settings have been fixed since the first implementation and no run has varied them. Each can be checked from the existing stage-1 checkpoint with a few short stage-2 runs.

Guard budget, 16 (`max_guards`). It must leave room above greedy's count, at most 11 on a training gallery, yet stay tight enough that running out means something. *To check:* log how often episodes end at the budget, per tier and stage. A budget that is never hit is not binding, and one hit often in stage 1 is too tight. Then train with 12, 16 and 20 and compare the validation score, solved rate and guard ratio. Three things depend on it and must move with it: the observed budget fraction $k/16$, the stage-1 scaffold sizing of section 4.6, and the room for spares below. Because the observation changes, stage 1 has to be retrained too.

Pre-filled starts, 4 spares on 40% of episodes. The policy's own solutions leave at most a few free guards (none for v11, table 4), far fewer than the 4 it practises on, and a fixed 4 never shows it a state with one stray guard. *To check:* short stage-2 runs from the same checkpoint with spares fixed at 2 and 4 and drawn uniformly from 1–4, and with 20, 40 and 60% pre-filled starts. Compare the validation score, the free guards left in solutions from zero guards, and the share of DEL actions that hit a free guard. More than 4 spares needs a larger budget, since $11 + 5$ already reaches 16. The two settings are `presaturate_extra` in `env.py` and `--presaturate`.

The same procedure applies to the other fixed constants: the ratchet's 1% dip allowance, the two-refusal limit and the reflex scaling of section 4.4.

5.2 Is a relocation move needed?

A MOVE mode would select the guard nearest a point and displace it by a bounded continuous vector. Delete-then-add already relocates a guard at zero net reward whenever the deletion passes the ratchet, so a dedicated move pays only if the policy's solutions can be improved by moves it cannot make today. That can be measured before building anything, on the best checkpoint's test solutions:

1. **Free guards left.** Remove free guards from each final solution until none remains. If almost none are left, deletion has nothing more to do, and the remaining gap to greedy is placement. For v11 none are left on any tier, so this check is already answered and the probe below is the next measurement.
2. **A one-move probe.** For each solution, try moving each guard to each of greedy's candidate points, and after each move apply the prune rule. Count the galleries where some single move frees a guard, and the guard ratio after the best move. Repeat with two moves. If a meaningful share of galleries improves, relocation would pay; if almost none do, a MOVE action would double the continuous action without a payoff.
3. **Does the policy relocate already?** Count DEL steps followed directly by an ADD in the trained policy's episodes, and whether those episodes end with fewer guards than they would without the pair.

If the probe says yes, the design is cheap to add to this architecture:

- **A displacement head.** MOVE gets its own cell map, as DEL has, masked to the guards' cells, and a second pair of Beta maps per cell gives a bounded displacement for the guard selected in that cell, and the log-probability factorises as in section 3.3.
- **Reward.** The potentials already value a move correctly: coverage pays $w_{\text{cov}}(f(S') - f(S))$, the redundancy potential pays $w_{\text{prune}}(R(s) - R(s'))$ when the move frees a neighbour, and the guard count does not change. No new reward term is needed.

- **Ratchet and masks.** A move should face the same floor as a deletion, and be masked below $c^\dagger$ like DEL, since a move is a deletion and an addition in one.

Other directions from the curriculum design remain open: replacing the fixed regularity penalties with a per-step constraint solved by N-P3O or CPO [2, 11]; sampling hard galleries in proportion to their recent failure rate; and, to claim a gap to the optimum rather than to greedy, an exact or near-exact solver [5, 15] on a sample of galleries.

References

[1] Mikkel Abrahamsen, Anna Adamaszek, and Tillmann Miltzow. The art gallery problem is $\exists\mathbb{R}$-complete. In *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 65–73, 2018.

[2] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 22–31, 2017.

[3] Po-Wei Chou, Daniel Maturana, and Sebastian Scherer. Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 834–843, 2017.

[4] Václav Chvátal. A combinatorial theorem in plane geometry. *Journal of Combinatorial Theory, Series B*, 18(1):39–41, 1975.

[5] Marcelo C. Couto, Pedro J. de Rezende, and Cid C. de Souza. An exact algorithm for minimizing vertex guards on art galleries. *International Transactions in Operational Research*, 18(4):425–448, 2011.

[6] Stephan Eidenbenz, Christoph Stamm, and Peter Widmayer. Inapproximability results for guarding polygons and terrains. *Algorithmica*, 31(1):79–113, 2001.

[7] Uriel Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4): 634–652, 1998.

[8] Steve Fisk. A short proof of chvátal's watchman theorem. *Journal of Combinatorial Theory, Series B*, 24(3):374, 1978.

[9] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the International FLAIRS Conference*, volume 35, 2022.

[10] D. T. Lee and Arthur K. Lin. Computational complexity of art gallery problems. *IEEE Transactions on Information Theory*, 32(2):276–282, 1986.

[11] Joonho Lee, Lukas Schroth, Victor Klemm, Marko Bjelonic, Alexander Reske, and Marco Hutter. Evaluation of constrained reinforcement learning algorithms for legged locomotion. *arXiv preprint arXiv:2309.15430*, 2023. URL https://arxiv.org/abs/2309.15430.

[12] Rosanne Liu, Joel Lehman, Piero Molino, Felipe Petroski Such, Eric Frank, Alex Sergeev, and Jason Yosinski. An intriguing failing of convolutional neural networks and the CoordConv solution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[13] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14(1): 265–294, 1978.

[14] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

[15] Davi C. Tozoni, Pedro J. de Rezende, and Cid C. de Souza. The quest for optimal solutions of the art gallery problem: A practical iterative algorithm. *ACM Journal of Experimental Algorithmics*, 21:1–23, 2016.

[16] Zhuoheng Wang, Jinyin Zhou, and Qi Wu. Dribble master: Learning agile humanoid dribbling through legged locomotion. *arXiv preprint arXiv:2505.12679*, 2025. URL https://arxiv.org/abs/2505.12679.

[17] Jimmy Wu, Xingyuan Sun, Andy Zeng, Shuran Song, Johnny Lee, Szymon Rusinkiewicz, and Thomas Funkhouser. Spatial action maps for mobile manipulation. In *Robotics: Science and Systems (RSS)*, 2020.